

Linux cron persistence investigation

Authored by Argyro Ouzounoglou. Public portfolio edition.

The internal hunt identifier is removed. Existing client/host masking is retained.

No malicious cron persistence was identified within the available telemetry.

One namespace lacked cron telemetry; the result is limited by that visibility gap.

9 MARCH 2026

THREAT HUNT CASE STUDY

Yro Ouzounoglou

CRON-BASED PERSISTENCE AND SUSPICIOUS COMMAND EXECUTION IN LINUX ENVIRONMENTS

EXECUTIVE SUMMARY

This threat hunt investigated the **potential abuse of Linux cron jobs as a persistence mechanism across multiple client namespaces.**

The investigation analyzed cron execution telemetry collected through **Elastic SIEM over a 30-day period** (Feb 3, 2026 – Mar 5, 2026). The analysis focused on identifying suspicious command execution patterns including downloader activity, encoded payload execution, and execution from temporary directories.

Cron activity across the following namespaces was analyzed:

- CLIENT_1
- CLIENT_2
- CLIENT_3
- CLIENT_4
- CLIENT_5

The investigation identified **several recurring cron tasks associated with legitimate infrastructure automation, including backup synchronization, monitoring agents, system maintenance tasks, and log management operations.**

No indicators of malicious persistence mechanisms, unauthorized cron job creation, or suspicious command execution patterns were identified.

PREPARE: DEFINE THE HUNT

Hunt Information	Details
Hypothesis	Adversaries may have established persistence on Linux hosts by creating or modifying cron jobs that execute suspicious commands (downloaders, shells, encoded payloads) and/or run from temporary memory-backed directories.
Threat Hunter Name	Yro Ouzounoglou
Date	Feb 23, 2026
Requestor	L2
Timeframe for hunt	Feb 3, 2026 – Mar 5, 2026
Threat Hunt Type	Structured Hunt (TTP-based)

SCOPING WITH THE ABLE METHODOLOGY

Field	Input
Actor	-
Behavior	Cron-based persistence (creation/ modification of cron jobs) + suspicious execution patterns (download/execute, encoded payloads, temp-path execution)
Location	Linux servers/VMs in scope
Evidence	Elastic SIEM telemetry including cron execution logs (CRON/crond), system.syslog events, and namespace-specific datasets capturing scheduled command execution activity on Linux hosts.

MITRE ATT&CK MAPPING

- T1053.003 – Scheduled Task/Job: Cron

Supporting (only if observed in command lines):

- T1059 – Command and Scripting Interpreter (bash, sh, python)
- T1105 – Ingress Tool Transfer (curl/wget downloading payloads)

BASELINE DEFINITION – LINUX CRON ACTIVITY

Cron telemetry was analyzed over a **30-day period** using **Elastic SIEM**.

Query used to establish baseline:

```
process.name: ("CRON" OR "crond") AND (message:*CMD* AND *certbot*)
```

Analysis of the telemetry identified a consistent and **expected cron task** related to **Let's Encrypt certificate renewal**.

Observed command:

The image shows a screenshot of the Elastic SIEM interface. At the top, there is a header bar with a blue icon and the word "message" in white. Below this, a text box explains that the message field contains log messages optimized for viewing in a log viewer. A section titled "Examples" follows, showing a sample log entry for a cron job. The entry is a multi-line command: "(root) CMD (test -x /usr/bin/certbot -a \! -d /run/systemd/system && perl -e 'sleep int(rand(43200))' && certbot -q renew --no-...". To the right of the command, it says "100%" and has expand/collapse icons. A blue progress bar is shown below the command. At the bottom, it states "Calculated from 100 sample records."

Baseline Characteristics

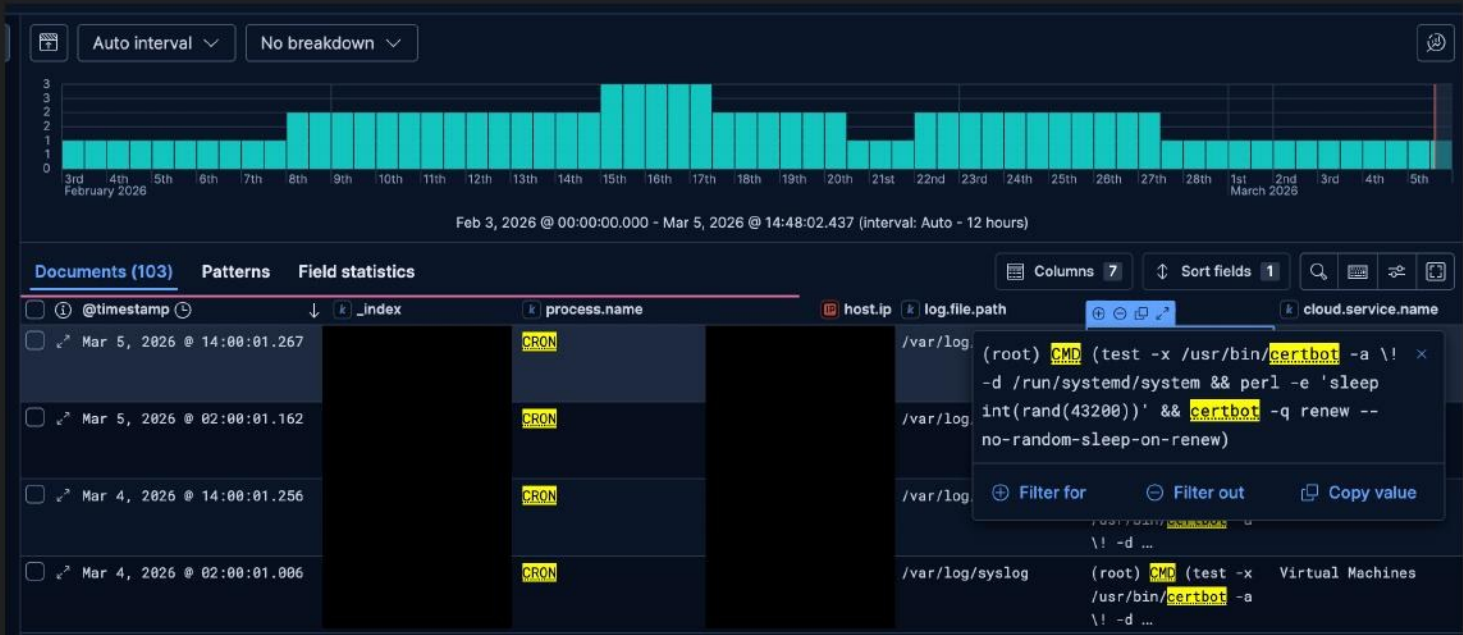
Attribute	Baseline Observation
User	root
Binary	/usr/bin/certbot
Execution Pattern	Periodic renewal job
Randomized Delay	perl sleep rand(43200) used to stagger renewal
Execution Location	/usr/bin/ system path
Network Behavior	None visible in cron message
Temporary Directory Execution	Not observed

Baseline Classification

Legitimate system automation task (Let's Encrypt certificate renewal).

No indicators of malicious persistence were identified in this cron activity.

Figure 1 – Cron activity baseline (last 30 days)



Baseline Scope

The baseline presented in this section reflects cron activity observed within the **CLIENT_1** namespace.

Because cron usage varies significantly between environments, baseline behavior **cannot be generalized across client namespaces.**

Therefore, baseline analysis was performed independently for each environment during the investigation.

EXECUTE: START THE HUNT (ELASTIC KQL)

CLIENT_1

Following baseline establishment, the execution phase focused on analyzing scheduled cron activity within the **CLIENT_1** client environment, specifically within the dataset:

- XXXXXXXXXXXX

The investigation began with the **following query** to enumerate cron command executions:

```
process.name: ("CRON" OR "crond") AND message:*CMD*
```

This query was executed across the **CLIENT_1** namespace and returned approximately **18,000** cron-related events **per host** during the timeframe:

- Feb 3, 2026 @ 00:01:25 - Mar 5, 2026 @ 15:41:43

The analysis was performed across the following production hosts:

Host-01
Host-02
Host-03
Host-04

Cross-Region Backup Synchronization

One of the most prominent **cron activities** observed across **all hosts was automated backup replication using rsync**, executed by the user:

- user01

The following **command pattern** was consistently observed:

```
rsync -a --delete /backup/dump_* user01@host[geo]:/backup/[geo]/
```

This job **synchronizes backup files** stored locally in:

```
/backup/dump_*
```

with **remote infrastructure nodes** in other regions.

- The **-a** flag ensures files are **copied while preserving permissions and timestamps**
- The **--delete** option guarantees that **files removed from the source server are also removed from the destination**, maintaining identical backup states across locations.

Across all hosts, this replication task **executed 24 times per day**, approximately **once per hour**, with slight variations in execution minutes depending on the region.

Observed replication patterns include:

Host	Replication Target
Host-01	XX and XX regions
Host-02	XX and XX regions
Host-03	XX and XX regions
Host-04	XX and XX regions

This behavior indicates a **cross-region backup redundancy mechanism**, ensuring that backup data is replicated between infrastructure nodes in these regions.

This pattern was consistently observed **across all environments**, suggesting a standardized infrastructure design for **backup redundancy and disaster recovery**.

OBSERVED CRON TASKS

System Monitoring and Maintenance Tasks

In addition to backup replication, several cron jobs related to **system monitoring** and **maintenance** were identified.

One common monitoring **task collects system and container resource statistics**:

```
{ date free -m docker stats --no-stream df -m | egrep "^/dev" } >> /var/log/docker-  
stats.log
```

This task records **system memory usage**, **Docker container statistics**, and **disk usage information**, appending the results to a monitoring log. Its purpose is to track infrastructure resource utilization over time.

Docker Storage Monitoring

Additional cron jobs monitor **Docker storage** usage through the following commands:

```
du -scm /var/lib/docker/volumes/* >> /var/log/docker-volumes-size.log
```

```
du -scm /var/lib/docker/* >> /var/log/docker-volumes-size.log
```

```
date >> /var/log/docker-volumes-size.log
```

These tasks **calculate the disk space used by Docker volumes and directories** and append the **results to a log file** with timestamps.

This allows **administrators to monitor container storage growth and detect potential storage issues.**

Log Rotation

Log maintenance tasks were also observed, including:

```
/usr/sbin/logrotate -s /var/lib/logrotate/omsagent-status /etc/logrotate.d/  
omsagent*
```

```
/usr/sbin/logrotate /etc/logrotate.d/scxagent --state /var/opt/microsoft/scx/log/scx-  
logrotate.status
```

These jobs **rotate log files generated by monitoring agents** used within the environment, ensuring logs **do not grow indefinitely** and consume excessive disk space.

System Performance Monitoring

The following task was observed **collecting system activity metrics**:

```
command -v debian-sa1 > /dev/null && debian-sa1 1 1
```

This command is part of the **sysstat monitoring utilities**, which collect CPU, memory, and system performance statistics.

Filesystem Maintenance

Filesystem integrity maintenance tasks were also identified:

```
test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
```

```
test -e /run/systemd/system || SERVICE_MODE=1 /usr/lib/x86_64-linux-gnu/  
e2fsprogs/e2scrub_all_cron
```

These commands perform **scheduled filesystem integrity checks** to ensure disk health and prevent corruption.

Standard Cron Maintenance Tasks

The system also executes **standard Linux maintenance scripts** using run-parts:

```
cd / && run-parts --report /etc/cron.hourly
```

```
test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily }
```

```
test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly }
```

```
test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly }
```

These tasks trigger **operating system maintenance scripts** that run on hourly, daily, weekly, and monthly schedules.

BASELINE OBSERVATIONS

Across all analyzed hosts, cron activity fell into the following operational categories:

Category	Example Task
System monitoring	debian-sa1
Log management	logrotate
System maintenance	run-parts
Filesystem maintenance	e2scrub_all
Resource monitoring	Docker disk usage checks
Backup management	/root/backup-*
Cross-region replication	rsync synchronization

Additionally, the **rsync** synchronization tasks were consistently **executed under the user01** account across all namespaces and regions. This indicates that backup replication is performed through a **dedicated operational account rather than the root user, which is a common practice in cloud-based infrastructure environments.**

ADDITIONAL DATASET VALIDATION – SYSTEM.SYSLOG

To further validate the observed cron activity, the investigation was expanded to the **system.syslog** dataset, which records system service activity and scheduled task execution logs generated by Linux services such as cron.

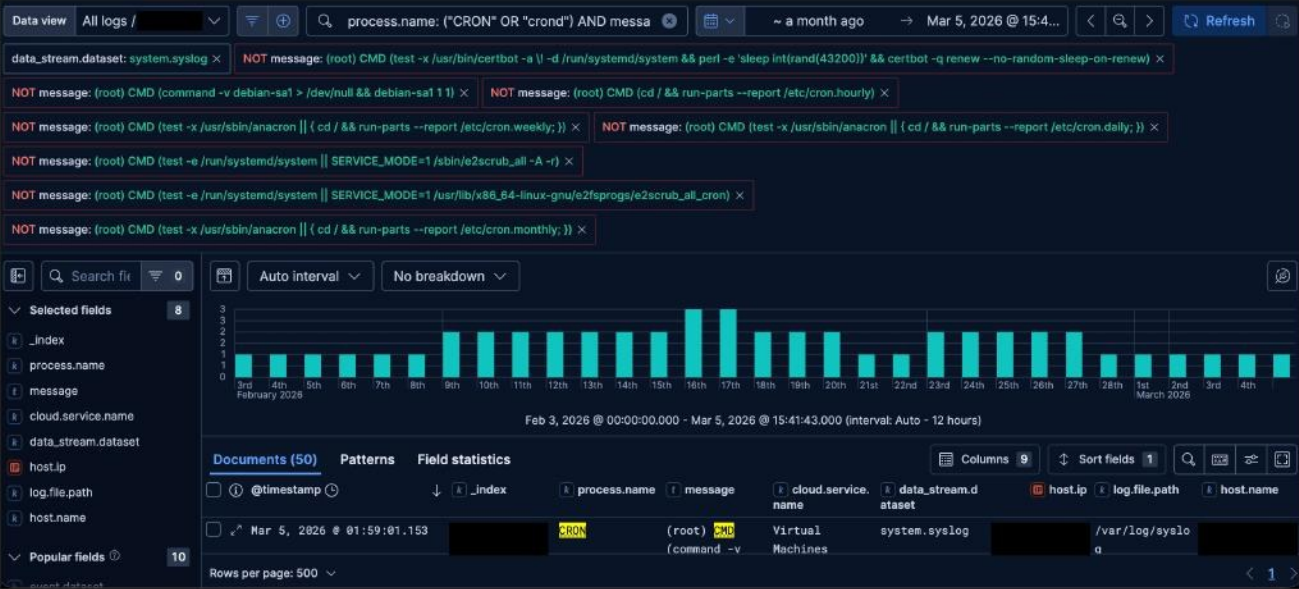
Analysis using the same detection logic

```
process.name: ("CRON" OR "crond") AND message:*CMD*
```

revealed **consistent cron execution patterns matching those previously identified in the XXXXX dataset.**

The commands observed within system.syslog logs correspond to the **same legitimate system maintenance and monitoring tasks**, including:

- system performance monitoring (debian-sa1)
- scheduled operating system maintenance (run-parts)
- filesystem integrity checks (e2scrub_all)
- log rotation operations (logrotate)
- automated certificate renewal (certbot)



CLIENT_2

The **CLIENT_2** namespace corresponds to **infrastructure supporting maritime operations**.

Systems within this environment appear to be associated with monitoring infrastructure components based on hostname patterns, likely related to Zabbix monitoring services.

The investigation focused on the following dataset:

- system.syslog

OBSERVED CRON TASKS

Analysis of the cron telemetry revealed **several recurring scheduled tasks associated with system maintenance, monitoring, and enterprise management tooling**.

The most prominent commands observed include:

PHP Session Maintenance

One of the most frequently observed tasks **performs PHP session cleanup**:

```
[ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; f
```

This task **removes expired PHP session files generated by web applications**.

PHP sessions are typically stored within directories such as:

```
/var/lib/php/sessions
```

Without periodic cleanup, these files could accumulate and **consume excessive disk space**.

The conditional logic ensures the script executes **only when systemd timers are not managing the same task, preventing duplicate execution**.

This behavior represents **standard maintenance activity on Linux systems running PHP-based services**.

Enterprise Endpoint Management Tasks

Cron telemetry also revealed scheduled tasks associated with the **ManageEngine Unified Endpoint Management (UEMS) agent**, located under:

```
/usr/local/manageengine/uems_agent/
```

Observed commands include:

```
/usr/local/manageengine/uems_agent/bin/dcagentupgrader
```

```
/usr/local/manageengine/uems_agent/bin/dcuiactionhandler -run upload-logs-root
```

These tasks are responsible for:

- endpoint agent updates
- log collection and upload
- endpoint telemetry reporting

The presence of these tasks indicates that the **infrastructure is integrated with enterprise endpoint management tooling**, which is commonly used to manage systems across distributed environments.

Filesystem Integrity Maintenance

Another scheduled task performs **filesystem maintenance operations** using the following command:

```
test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
```

This task executes the **e2scrub_all** utility, which performs integrity checks on ext-based Linux filesystems.

The purpose of this operation is to detect **filesystem inconsistencies and prevent potential corruption issues**.

These checks are typically executed on a scheduled basis as part of operating system maintenance.

Time Synchronization Log Rotation

A scheduled task associated with **time synchronization services** was also observed:

```
/usr/libexec/ntpsec/rotate-stats
```

This task rotates statistics logs generated by the **NTP time synchronization service**, ensuring that log files do not grow indefinitely.

Maintaining accurate system time is critical for logging accuracy, monitoring systems, and security auditing.

BASELINE OBSERVATIONS

Across the analyzed hosts, cron activity fell into the following operational categories:

Category	Example Task
System monitoring	Zabbix monitoring infrastructure
Log management	NTP statistics rotation
System maintenance	PHP session cleanup
Filesystem maintenance	e2scrub_all
Endpoint management	ManageEngine UEMS agent operations

The commands were executed by the **root user** and originated from expected system or vendor directories, including:

/usr/lib/php/
/usr/local/manageengine/
/sbin/
/usr/libexec/

CLIENT_3

The **CLIENT_3** namespace corresponds to **infrastructure supporting maritime operations**, with systems deployed across ship-connected environments and associated operational monitoring platforms.

The presence of hosts such as:

- Host_A
- Host_B

suggests **geographically distributed infrastructure**, potentially supporting vessel operations and monitoring systems across multiple regions.

Following the analysis performed across the CLIENT_1 and CLIENT_2 namespaces, the investigation continued with the examination of cron execution activity within the **CLIENT_3** namespace.

OBSERVED CRON TASKS

The most prominent commands observed include the following.

Pulsar Monitoring Agent Execution

One of the most frequently observed cron tasks executes a NodeJS-based monitoring agent:

```
node /home/service_account/pulsar-agent
```

Additionally, a similar command was observed executing within an application directory:

```
cd /home/user_home/pulsar2.1-client; node .
```

These commands indicate the execution of a **Node.js-based telemetry or monitoring agent**, likely responsible for collecting operational metrics or application telemetry from the system.

The execution of these agents via cron suggests that the system periodically restarts or executes the monitoring agent to ensure continued operation and telemetry collection.

System Performance Monitoring

A standard Linux monitoring command was also observed:

```
command -v debian-sa1 > /dev/null && debian-sa1 1 1
```

This command belongs to the **sysstat monitoring** utilities, which collect system performance metrics including CPU usage, memory utilization, and disk activity.

The command **first verifies the presence of the monitoring utility and then executes the system activity collector**.

This represents standard system monitoring behavior.

Standard Cron Maintenance Tasks

Another scheduled task executes the default Linux cron hourly script directory:

```
cd / && run-parts --report /etc/cron.hourly
```

This task triggers the execution of scripts located within the **/etc/cron.hourly** directory.

These scripts typically perform various operating system maintenance functions such as log cleanup, monitoring tasks, or system housekeeping operations.

BASELINE OBSERVATIONS

Across the analyzed hosts, cron activity fell into several operational categories:

Category	Example Task
System monitoring	debian-sa1
Application monitoring	Pulsar NodeJS telemetry agent
System maintenance	run-parts cron execution

The commands were executed under the following users:

- service_account
- root

Execution paths originated from expected application or system directories, including:

/home/service_account/

/home/user_home/

/usr/bin/

/etc/cron.hourly

CLIENT_4

Analysis of cron telemetry within the **CLIENT_4** namespace revealed recurring **scheduled tasks associated with application monitoring agents** and **Linux system maintenance operations**.

OBSERVED CRON TASKS

The most frequently observed commands include the following.

Pulsar Telemetry Agent Execution

Similar to the behavior observed within the CLIENT_3 namespace, cron telemetry revealed the execution of a NodeJS-based monitoring agent:

```
cd /home/service_account/pulsar2.1-client; node .
```

This command launches a NodeJS-based application located within the user directory.

The pulsar agent likely functions as a telemetry or monitoring component, responsible for collecting operational metrics or system telemetry from the host environment.

Execution through cron ensures that the monitoring agent runs continuously or is periodically restarted if necessary.

BASELINE OBSERVATIONS

Category	Example Task
System monitoring	debian-sa1
Application monitoring	Pulsar NodeJS telemetry agent
System maintenance	run-parts
Filesystem maintenance	e2scrub_all
Network monitoring	WAN monitoring scripts

Commands were executed under the following users:

- service_account
- root

Execution paths originated from expected system or application directories, including:

/home/service_account/
/usr/bin/
/etc/cron.*
/root/wanmon/

CLIENT_5

The investigation was further extended to the **CLIENT_5** namespace, which corresponds to maritime infrastructure associated with vessel-connected environments.

Unlike other analyzed namespaces, **cron telemetry within the CLIENT_5 environment did not produce any observable cron execution events**

The **absence of cron execution telemetry** may indicate either an alternative task scheduling mechanism (e.g., systemd timers) or incomplete telemetry collection for cron-related logs.

From a threat hunting and detection engineering perspective, this represents a **potential telemetry visibility gap** that should be **reviewed** to ensure that scheduled task activity can be properly monitored within the environment.

GLOBAL HUNT ASSESSMENT

Across all analyzed namespaces, cron activity was consistent with **legitimate infrastructure automation, system monitoring, and maintenance operations.**

The investigation identified several recurring scheduled tasks including **backup synchronization, monitoring agents, system maintenance scripts, and log management operations.**

No indicators of malicious cron-based persistence mechanisms, unauthorized scheduled task creation, or suspicious command execution patterns were identified during the analyzed timeframe.

The CLIENT_5 namespace did not produce observable cron execution telemetry, which may indicate alternative scheduling mechanisms or a telemetry visibility gap that should be reviewed.

Overall, the analyzed cron activity reflects normal operational automation and infrastructure monitoring behavior across the investigated environments.

DETECTION LOGIC

The objective of this hunt was to identify potential cron-based persistence mechanisms and suspicious command execution patterns that could indicate adversary activity on Linux systems.

The following detection logic can be used to **convert the hunting queries into production detection rules.**

Initial Detection Criteria

The following conditions may trigger a detection alert:

Suspicious Cron Execution

Detection query example:

```
process.name: ("CRON" OR "crond") AND message:*CMD*
```

Alert conditions:

- execution of commands containing curl, wget, nc, bash -c, sh -c, python -c
- execution of commands referencing base64 decoding
- execution of commands referencing network sockets (/dev/tcp)
- execution of binaries from temporary directories

Example suspicious patterns:

```
message:*curl*  
message:*wget*  
message:*nc*  
message:*base64*  
message:*/tmp/*  
message:*/dev/shm/*  
message:*bash -c*  
message:*sh -c*
```

Threshold Indicators

The following behaviors may indicate malicious activity:

- cron job creation or modification outside normal maintenance windows
- cron jobs executing download-and-execute commands
- cron tasks executing binaries from memory-backed directories
- repeated execution of encoded commands

Recommended alert thresholds:

Condition	Alert Type
cron executing curl/wget	HIGH
cron executing from /tmp	HIGH
cron executing base64 payload	HIGH
new cron task creation	MEDIUM

Refinements After Review

During the hunt, **several legitimate cron tasks generated** results but were determined to be benign.

Examples include:

- system monitoring tasks (debian-sa1)
- log rotation (logrotate)
- filesystem maintenance (e2scrub_all)
- backup synchronization (rsync)
- monitoring agents (Pulsar telemetry)

To **reduce false positives**, the detection rule should exclude:

```
message:*debian-sa1*
```

```
message:*logrotate*
```

```
message:*run-parts*
```

```
message:*e2scrub_all*
```

```
message:*rsync*
```

```
message:*certbot*
```

ACT: FINDINGS & RESPONSE

A structured threat hunt was conducted across **multiple client namespaces** to identify potential cron-based persistence mechanisms and suspicious command execution patterns.

Cron telemetry across all analyzed namespaces was examined using **Elastic SIEM queries designed to detect scheduled task abuse and malicious command execution.**

The investigation **did not identify any indicators of malicious persistence or unauthorized cron activity.**

Observed cron executions were associated with **legitimate infrastructure automation, monitoring agents, and system maintenance tasks.**

Findings	Description
No malicious cron jobs identified	All observed cron executions correspond to legitimate operational tasks
Cross-region backup automation	Backup replication tasks using rsync observed across CLIENT_1 infrastructure
Monitoring agent telemetry	NodeJS-based Pulsar monitoring agents observed in maritime environments
Telemetry gap identified	CLIENT_5 namespace did not generate cron execution telemetry

Namespace	Cron Activity	Suspicious Activity	Conclusion
CLIENT_1	YES	NONE	Legitimate automation
CLIENT_2	YES	NONE	Legitimate monitoring
CLIENT_3	YES	NONE	Monitoring telemetry agents
CLIENT_4	YES	NONE	Infrastructure monitoring
CLIENT_5	NO	NONE	Telemetry gap

KNOWLEDGE: LESSONS LEARNED

What Worked Well

- Baseline establishment **allowed rapid identification of legitimate cron behavior**
- Cross-namespace comparison helped **validate infrastructure patterns**
- **Elastic SIEM telemetry** provided sufficient **visibility** into scheduled task execution

Improvements

Future hunts could be enhanced by:

- monitoring cron file modifications
- correlating cron execution with process execution logs
- expanding telemetry collection for environments where cron logging visibility is limited

The hunting logic can be converted into a **detection rule for identifying suspicious cron executions involving:**

- network download utilities
- encoded payload execution
- execution from temporary directories